

Assignment 1

Abdallahman Mohammed
abmm22@student.bth.se

Question 1: Optimization Using Gradient Descent

Introduction

This report shows how gradient descent can be used to minimize a simple two variable function. The aim is to find the values of x and y that make the function as small as possible.

Function and Partial Derivatives

The function is:

$$f(x, y) = (x - 3)^2 + (y + 2)^2$$

Partial derivative of the function with respect to x :

$$\begin{aligned}\frac{df}{dx} &= \frac{d}{dx}((x - 3)^2) + \frac{d}{dx}((y + 2)^2) \\ \frac{d}{dx}((x - 3)^2) &= \text{Using chain rule,} \quad \frac{d}{dx}((y + 2)^2) = \text{Constant} \\ \frac{df}{dx} &= 2(x - 3) + 0 \\ \frac{df}{dx} &= 2(x - 3)\end{aligned}$$

Partial derivative of the function with respect to y :

$$\begin{aligned}\frac{df}{dy} &= \frac{d}{dy}((x - 3)^2) + \frac{d}{dy}((y + 2)^2) \\ \frac{d}{dy}((x - 3)^2) &= \text{Constant,} \quad \frac{d}{dy}((y + 2)^2) = \text{Using chain rule} \\ \frac{df}{dy} &= 0 + 2(y + 2) \\ \frac{df}{dy} &= 2(y + 2)\end{aligned}$$

Gradient Descent Implementation

Gradient descent was implemented to minimize the function $f(x, y)$. The algorithm started from an initial point and then updated the values of x and y step by step over a fixed number of iterations. In each iteration the current derivatives were used to adjust the parameters in the direction that reduces the function value. In this implementation the learning rate was set to 0.1 and the number of iterations was set to 30. These values were chosen because they give stable and efficient convergence for this simple quadratic function. A learning rate of 0.1 is large enough to make training reasonably fast but small enough to avoid taking steps that are too big.

The number of iterations was set to 30 because this was sufficient for the algorithm to move very close to the minimum point. After 30 iterations, the result obtained from the implementation was approximately:

$$x = 2.9963$$

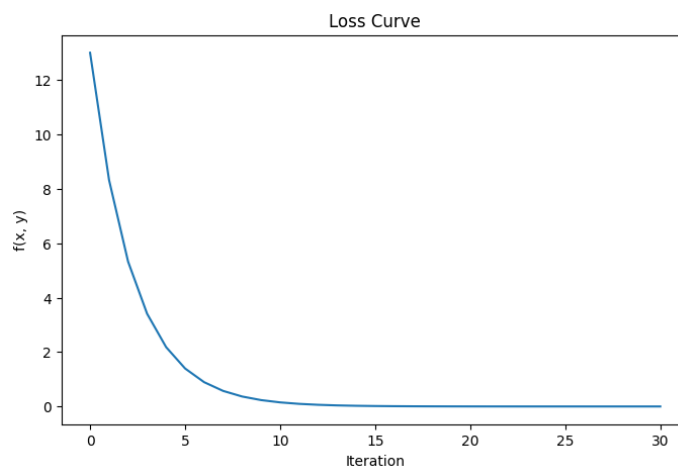
$$y = -1.9975$$

$$f(x, y) = 2 \cdot 10^{-5}$$

The final values are very close to the true minimum at $(3, -2)$ where $f(3, -2) = 0$. The final value is not exactly zero because gradient descent stops after a limited number of iterations. But it is close enough to show that the implementation worked correctly.

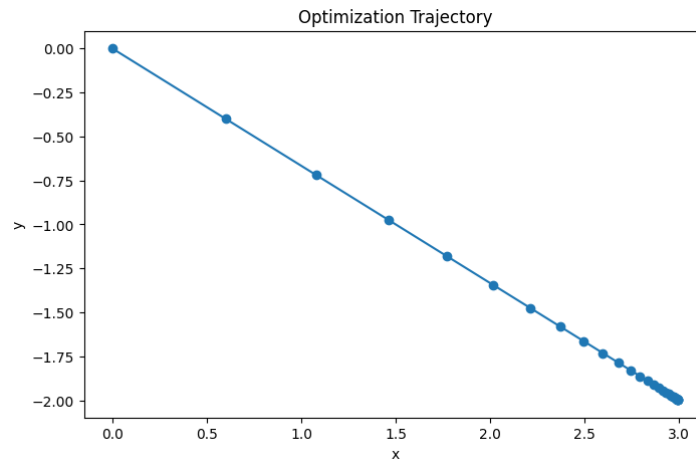
Loss Curve

The loss curve shows that the loss is relatively large at the beginning because the starting point is far from the minimum. As the iterations progress the loss decreases quickly and then slowly approaches zero. This confirms that gradient descent is minimizing the function successfully. The curve also shows stable progress because the function value keeps decreasing without large jumps or instability.



Optimization Trajectory

The optimization trajectory shows how x and y change during the optimization process. Starting from the initial point the updates move gradually toward $(3, -2)$ which is the minimum of the function. The trajectory shows that both parameters move smoothly toward their best values. Near the minimum the updates become smaller because the gradient also becomes smaller.



Conclusion

Gradient descent successfully minimized the function and found values very close to the true minimum. The loss curve and optimization trajectory both show that the algorithm converged in a stable and correct way.

Question 2: Deep Learning for Image Classification

Introduction

Image classification is important in deep learning where a model learns to place images into predefined classes. In this project the CIFAR-100 dataset was used to train and evaluate several deep learning models for image classification. CIFAR-100 contains colour images from 100 classes and this makes the task more difficult than in datasets with fewer categories.

The main purpose of the experiment was to design a custom convolutional neural network and compare its performance with established deep learning architectures. The custom CNN was trained from scratch. The pre-defined models used in the comparison were ResNet-50, VGG-19, DenseNet-121 and EfficientNet-B0. The models were evaluated using training results and validation results and test accuracy and F1-score and confusion matrix-based metrics.

Custom Architecture Description

The custom CNN used in this project has eight convolutional layers. The model is organised into four convolutional blocks followed by a classifier. The input is an RGB image resized to 224×224 pixels.

Each block contains two convolutional layers. All convolutional layers use 3×3 filters with stride 1 and padding 1. After each convolutional layer batch normalization and ReLU activation are applied. At the end of each block max pooling with a 2×2 filter is used to reduce the size of the feature maps.

The number of channels increases after each block. The first block changes the input from 3 channels to 32 channels. The second block increases the number of channels to 64. The third block increases it to 128. The fourth block increases it to 256.

After the convolutional blocks adaptive average pooling reduces the feature maps to 1×1 . The result is then flattened and passed through a fully connected layer with 512 neurons. ReLU activation and dropout with a rate of 0.5 are used before the final output layer. The final layer has 100 output neurons with one neuron for each class in the CIFAR-100 dataset.

Training Configuration

All models were implemented in PyTorch. The custom CNN used ReLU as the activation function after each convolutional layer and in the classifier. ReLU was chosen because it is simple and efficient and widely used in image classification tasks.

The loss function used for all models was CrossEntropyLoss because the task is a multi class classification problem. The optimizer was Adam with a learning rate of 0.001.

Several regularization methods were used to improve generalization and reduce overfitting. These included batch normalization and dropout with a rate of 0.5 and data augmentation and early stopping.

The main hyperparameters were a batch size of 128, a maximum of 50 epochs, a learning rate of 0.001 and 4 workers for data loading. Mixed precision training was enabled. Although the maximum number of epochs was set to 50 early stopping with a patience of 5 was used.

Data Preprocessing

The CIFAR-100 dataset was loaded using `torchvision.datasets.CIFAR100`. All images were resized from 32×32 pixels to 224×224 pixels so they matched the input size expected by the pretrained models.

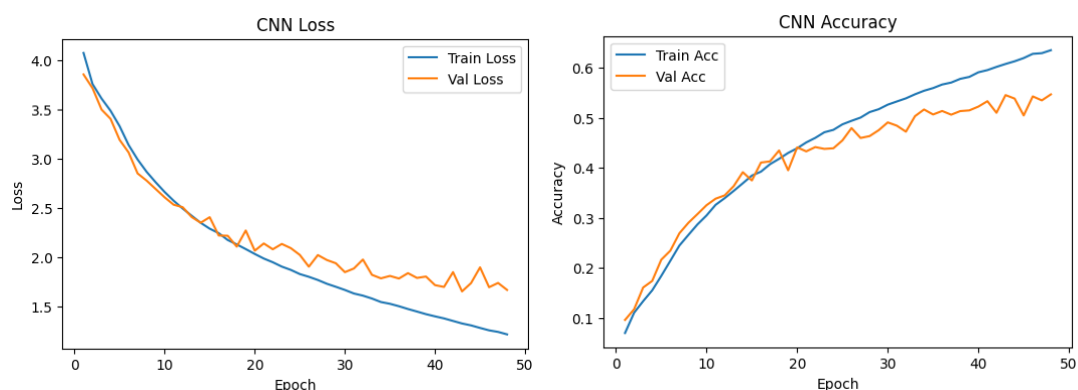
The images were also normalized using the ImageNet mean and standard deviation because the pre-trained models were originally trained on ImageNet. For data augmentation a random horizontal flip was applied only to the training set.

Training Analysis

Custom CNN

The custom CNN showed a steady learning trend during training. Training loss decreased from 4.0764 to 1.2154 while validation loss decreased from 3.8575 to 1.6673 before early stopping. Training accuracy increased from 6.95% to 63.52% while validation accuracy increased from 9.56% to 54.68%. The final test accuracy was 55.52%.

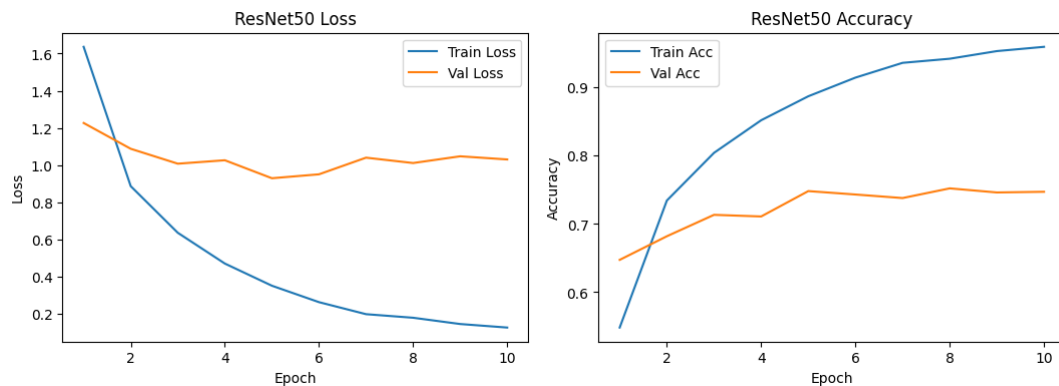
The loss and accuracy curves indicate that the model learned useful features from scratch. However a gap appeared between training and validation performance in the later epochs and this suggests mild overfitting. Even so validation accuracy continued to improve for a long period and this indicates that the model was still learning effectively.



ResNet-50

ResNet-50 learned much faster than the custom CNN. Training loss dropped rapidly from 1.6364 to 0.1275 while training accuracy increased from 54.81% to 95.97%. Validation accuracy reached around 75.22% and the final test accuracy was 74.13%.

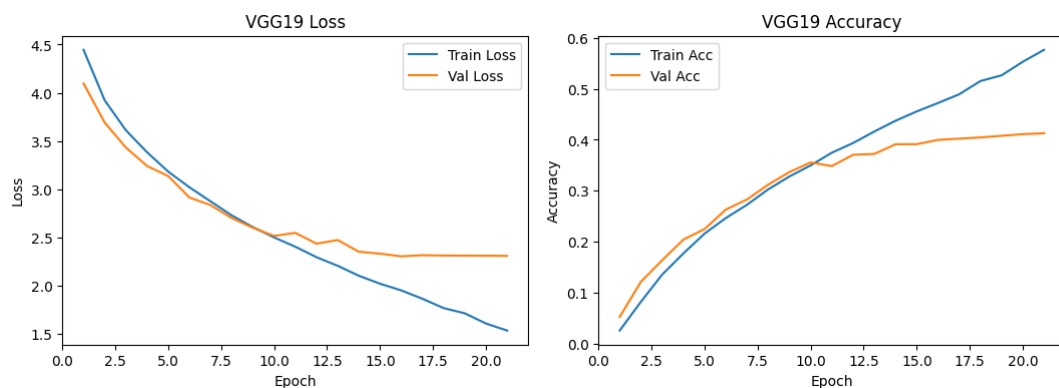
The training curves show that ResNet-50 learned very efficiently during the first few epochs. However after some point training loss kept decreasing while validation loss stopped improving consistently. This indicates overfitting in the later epochs. Early stopping was therefore useful because it prevented unnecessary extra training.



VGG-19

VGG-19 improved gradually but performed worse than the other pre-trained models. Training accuracy increased from 2.59% to 57.64% while validation accuracy increased from 5.28% to 41.28%. The final test accuracy was 41.05%.

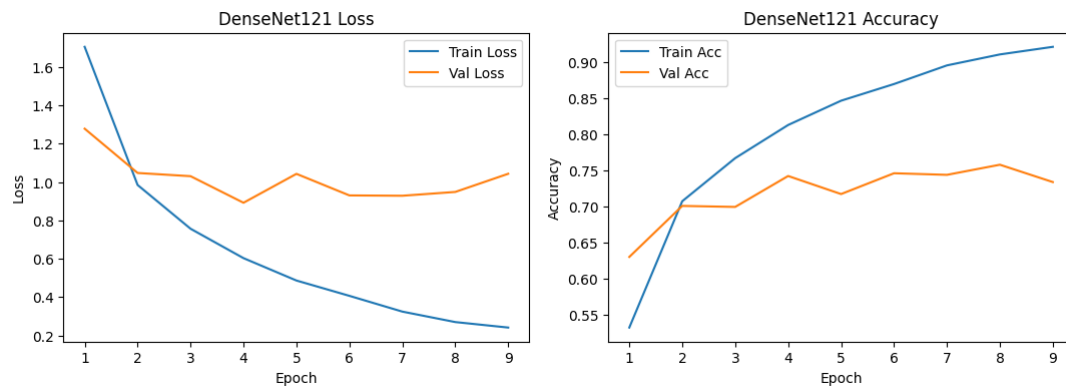
The learning curves show that VGG-19 did learn from the data but more slowly and less effectively than the other pre-trained networks. Validation performance improved only modestly, the large gap between training and validation performance suggests overfitting together with weaker generalization.



DenseNet-121

DenseNet-121 showed strong learning behaviour. Training accuracy increased from 53.24% to 92.10% while validation accuracy reached 75.78% before early stopping. The final test accuracy was 74.03%.

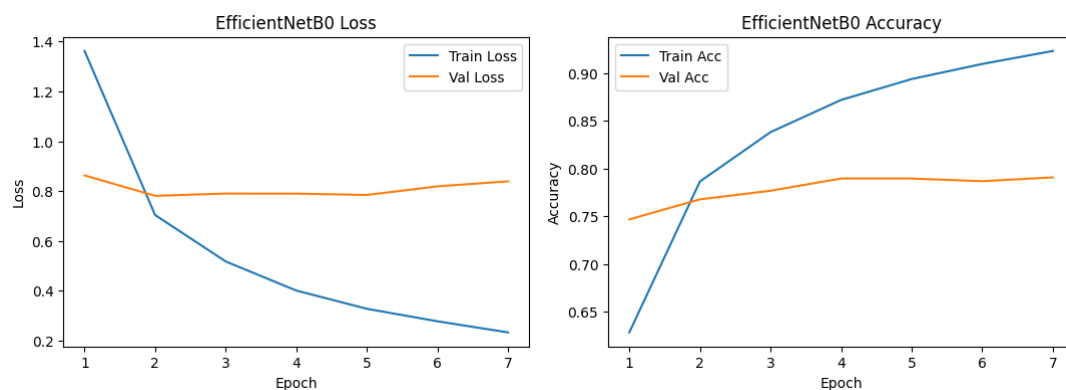
The loss and accuracy curves show rapid improvement during the first epochs. Later validation loss began to fluctuate while training loss kept decreasing and this suggests overfitting. Nevertheless DenseNet-121 achieved strong overall performance and generalized well to the test set.



EfficientNet-B0

EfficientNet-B0 produced the strongest overall result. Training accuracy increased from 62.85% to 92.31% while validation accuracy reached about 79.08%. The final test accuracy was 76.37%.

The validation curves remained strong and relatively stable and this indicates that EfficientNet-B0 achieved the best balance between learning and generalization. Although some overfitting can still be seen it was less problematic than in the other models.



Overfitting, underfitting, and mitigation strategies

Most models showed some overfitting. This was especially clear for ResNet-50 and DenseNet-121 where training performance continued to improve while validation performance became stable or fluctuated. The custom CNN also showed mild overfitting while VGG-19 showed weaker learning and poorer generalization.

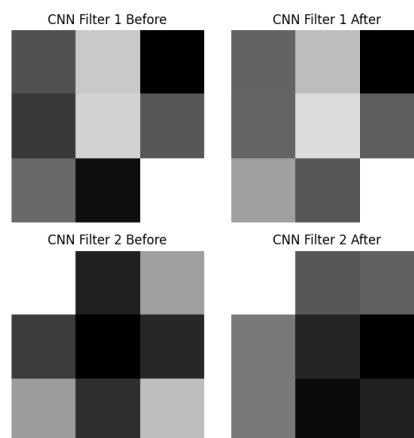
To reduce overfitting several methods were used. These included batch normalization and dropout and data augmentation and early stopping. Early stopping was especially important because it stopped training when validation loss did not improve for 5 consecutive epochs.

Weight Evolution Analysis

To analyse how the custom CNN changed during training two filters from the first convolutional layer were visualized before training and after training.

The filters appear as block like grayscale patterns because the first convolutional layer uses small 3×3 kernels. Before training the filters represent randomly initialized weights. After training small changes can be observed and this shows that the CNN updated its filters during learning.

These early convolutional filters are expected to learn simple low level image features such as edges and contrast differences and texture patterns. Later layers can then combine these simple features into more complex image representations.



Evaluation Metrics

Model	Accuracy	TP	TN	FP	FN	F1-score
CNN	0.5558	55.58	9855.58	44.42	44.42	0.554333
ResNet50	0.7409	74.09	9874.09	25.91	25.91	0.738865
VGG19	0.4103	41.03	9841.03	58.97	58.97	0.406964
DenseNet121	0.7403	74.03	9874.03	25.97	25.97	0.740967
EfficientNetB0	0.7633	76.33	9876.33	23.67	23.67	0.760493

Because CIFAR-100 is a multiclass classification task with 100 classes TP, TN, FP and FN were calculated using a one vs rest approach for each class and then averaged across all classes. Therefore these values are shown as averaged values rather than simple integer counts from one binary confusion matrix.

Model Comparison

The results show that EfficientNet-B0 was the best performing model and it achieved the highest test accuracy at 76.33% and the highest F1-score at 0.7605. ResNet-50, DenseNet-121 also performed very well and both achieved about 74% test accuracy. EfficientNet-B0 likely achieved the best results because it provides a strong balance between representation power and efficiency. Its architecture is more modern and more optimized than the older networks. ResNet-50 benefits from residual connections and these help optimization in deep networks. DenseNet-121 benefits from dense connectivity and feature reuse and this can improve information flow and learning efficiency.

The custom CNN achieved 55.58% accuracy and this was lower than the strongest pre-trained models. However it still represents a good result for a model trained entirely from scratch. This shows that the architecture could learn useful image features even though it could not match the performance of transfer learning. The custom CNN started from random weights and had to learn all features directly from CIFAR-100. In contrast the standard architectures started from ImageNet pretraining and this gave them a strong advantage.

VGG-19 performed the worst among the pre-trained models with 41.03% accuracy. One likely reason is that VGG-19 is an older and less parameter efficient architecture than ResNet and DenseNet and EfficientNet. It may also require different fine-tuning settings to achieve stronger performance. VGG-19 likely underperformed because it is deeper but less efficient and its architecture may not adapt as effectively under the same training setup.

Conclusion

This experiment compared a custom CNN trained from scratch with four established pretrained architectures on CIFAR-100. The custom achieved a test accuracy of 55.58% and this shows that it successfully learned meaningful visual features from scratch.

Among the compared models EfficientNet-B0 achieved the best overall performance followed closely by ResNet-50 and DenseNet-121. VGG-19 produced the weakest result among the pretrained architectures.

The results show that transfer learning with strong pretrained models clearly outperformed the custom CNN although the custom model still provided a useful and interpretable baseline. The training plots and filter analysis also show that the custom CNN learned progressively more meaningful image representations during training.